

Deep Learning – Homework#9-1

Keras 教學

本次作業將學習到：

- 1) 學習使用 Keras，它是用 Python 編寫的高級神經網絡 API (編成框架) 並且能夠在包括 TensorFlow 和 CNTK 在內的幾個較低級別的框架上運行。
- 2) 了解如何在構建深度學習算法

Keras 的目的是使深度學習工程師能夠快速建構和試驗不同的模型，正如 Tensorflow 比 Python 更高級的框架一樣，Keras 是一個更高級別的框架，能夠將想法付諸實踐，找到盡快找到好的模型的關鍵。但是，Keras 比底層框架更具限制性，因此您可以在 TensorFlow 中實現一些非常複雜的模型，而在 Keras 中則較為困難，話雖如此，常見的模型 Keras 內皆有收入。

在本練習中，將解決一個“多分類”的問題 (Multiple classification)，我們將在下面進行解釋，讓我們加載所需的軟件包並解決多分類的問題

首先，加載所需要使用的軟件包

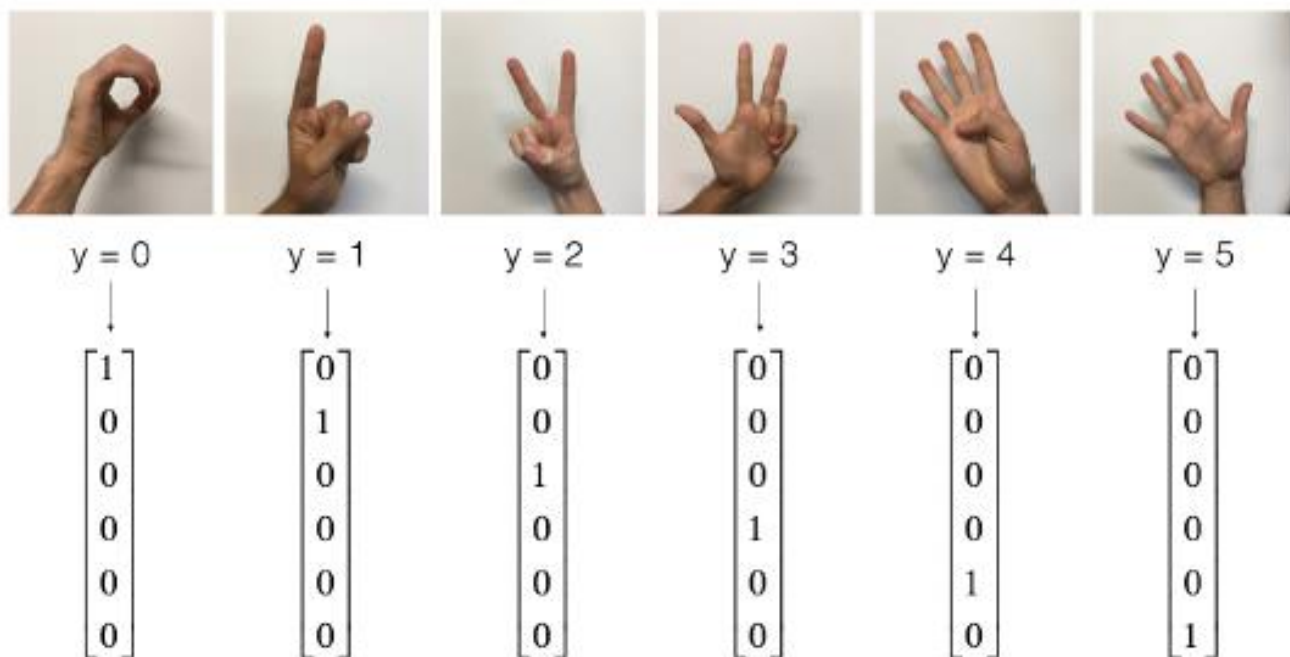
```
import numpy as np
#import tensorflow as tf
from keras import layers
from keras.layers import Input, Dense, Activation, ZeroPadding2D, BatchNormalization, Flatten, Conv2D
from keras.layers import AveragePooling2D, MaxPooling2D, Dropout, GlobalMaxPooling2D, GlobalAveragePooling2D
from keras.models import Model
from keras.preprocessing import image
from keras.utils import layer_utils
from keras.utils.data_utils import get_file
from keras.applications.imagenet_utils import preprocess_input
import pydot
from IPython.display import SVG
from keras.utils.vis_utils import model_to_dot
from keras.utils import plot_model, to_categorical
from kt_utils import *

import keras.backend as K
K.set_image_data_format('channels_last')
import matplotlib.pyplot as plt
from matplotlib.pyplot import imshow
```

注意：我們從 Keras 導入了許多功能。您可以直接在文件中直接調用它們來輕鬆使用它們，例如：
X=input(...) 或 X=ZeroPadding2D(...)。

1. 多分類問題:

本次的分類問題還是採用之前的 SIGN 資料集，主要分為 6 類型的集合，其標籤由 0~5 的數字表示如下圖一所示：



由於原始的資料標籤是 0~5，我們需要使用編碼 (One-hot encoder)，將標籤轉換成不同的二位元數列，分別表示一種標籤結果。

執行以下程式碼，載入資料集和標籤轉換二位元數列

```
X_train_orig, Y_train_orig, X_test_orig, Y_test_orig, classes = load_dataset()

# Normalize image vectors
X_train = X_train_orig/255.
X_test = X_test_orig/255.

# Reshape
Y_train = to_categorical(Y_train_orig.T) # to_categorical() , keras function
Y_test = to_categorical(Y_test_orig.T)

print ("number of training examples = " + str(X_train.shape[0]))
print ("number of test examples = " + str(X_test.shape[0]))
print ("X_train shape: " + str(X_train.shape))
print ("Y_train shape: " + str(Y_train.shape))
print ("X_test shape: " + str(X_test.shape))
print ("Y_test shape: " + str(Y_test.shape))
```

其中 load_dataset() 在提供的 kt_utils.py 內不需要修改，to_categorical 是 keras 本身提供的 one-hot encoder。

SIGN 數據集的詳細資料：

```
number of training examples = 1080
number of test examples = 120
X_train shape: (1080, 64, 64, 3)
Y_train shape: (1080, 6)
X_test shape: (120, 64, 64, 3)
Y_test shape: (120, 6)
```

訓練集圖片 1080 張、測試集圖片 120 張，每張圖片為 (64,64,3) 的形狀代表 3 原色通道，每個通道的像素為 64x64，圖片分為 6 種類型。

2. 建構 Keras 的模型:

Keras 非常適合快速原型製作，在很短的時間內，將能夠建立一個能夠獲得出色結果的模型。

以下是 Keras 中的模型示例：

```
def model(input_shape):
    # Define the input placeholder as a tensor with shape input_shape. Think of this as
    # your input image!
    X_input = Input(input_shape)

    # Zero-Padding: pads the border of X_input with zeroes
    X = ZeroPadding2D((3, 3))(X_input)

    # CONV -> BN -> RELU Block applied to X
    X = Conv2D(32, (7, 7), strides = (1, 1), name = 'conv0')(X)
    X = BatchNormalization(axis = 3, name = 'bn0')(X)
    X = Activation('relu')(X)

    # MAXPOOL
    X = MaxPooling2D((2, 2), name='max_pool')(X)

    # FLATTEN X (means convert it to a vector) + FULLYCONNECTED
    X = Flatten()(X)
    X = Dense(1, activation='sigmoid', name='fc')(X)

    # Create model. This creates your Keras model instance, you'll use this instance to
    # train/test the model.
    model = Model(inputs = X_input, outputs = X, name='HappyModel')

    return model
```

注意：Keras 對變量名使用的約定與我們之前在 numpy 和 TensorFlow 中使用的約定不同。在 TensorFlow 為了不同層的計算，與其在前進的每一步上創建和分配新的變量(例如：X、Z1、A1、Z2、A2 等等)；在 Keras 中，只需要使用 X = ... 將 X 賦予新值即可。換句話說，在正向傳播的每個步驟中，只是將計算中的最新值寫入相同的變量 X，唯一的例外是 “X_input”，我們將其分開並沒有覆蓋，因為我們最後需要它來創建 Keras 模型實例。

練習：實現一個 `SIGNModel()`， 這個任務比大多數任務更開放，我們建議首先要使用我們建議的體系結構來實現模型，然後再使用該模型作為初始模型來完成本任務的其餘部分，主動嘗試其他模型架構。例如，避免 `overfitting`，可以使用其他功能 `AveragePooling2D(...)`、`GlobalMaxPooling2D(...)`、`Dropout(...)` 等等。

```
def SIGNModel(input_shape):  
    """  
    Implementation of the SIGN.  
  
    Arguments:  
    input_shape -- shape of the images of the dataset  
  
    Returns:  
    model -- a Model() instance in Keras  
    """  
    ### START CODE HERE ###  
    # Feel free to use the suggested outline in the text above to get started,  
    # exercise (including the later portions of this notebook) once. The come b  
    # network architectures as well.  
    X_input = Input(shape=input_shape)  
    X = ZeroPadding2D(padding=(1, 1))(X_input)  
    X = Conv2D(8, kernel_size=(3,3), strides=(1,1))(X)  
    X = BatchNormalization(axis=3)(X)  
    X = Activation('relu')(X)  
    X = MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='valid')(X)
```

```
    X =  
    X =  
    X =  
    X =  
    X =  
  
    # FC  
    X =  
    Y1 =  
    Y =  
  
    model = Model(inputs = X_input, outputs = Y, name='SIGNModel')  
    ### END CODE HERE ###  
  
    return model
```

備註：第二的 `X` 重覆第一層 `X` 的特徵擷取，`ZeroPadding2D(...)` → `Conv2D(...)` → `BatchNormalization(...)` → `Activation(...)` → `Maxpooling2D(...)`，第三層全連接層總和特徵分類，第一步將特徵展平使用 `Flatten(...)`，第二步全連接層 `Dense(..)` 不使用任何激活代表線性模塊，第三步輸出經過 `One-hot decoder` 轉換，可以使用 `Activation(...)` 激活函數使用 'softmax'。

現在，您已經構建了一個描述模型的函數。為了訓練和測試該模型，Keras 中有四個步驟：

1) 定義模型如前面所述並引用它

```
SIGNModel = SIGNModel((64, 64, 3))
```

2) 編譯模型

```
SIGNModel.compile(optimizer = "sgd", loss = "categorical_crossentropy", metrics = ["accuracy"])
```

其中 optimizer 這邊採用最簡單的 sgd，loss 函數根據不同問題需要更換，這邊是多分類問題使用 categorical_crossentropy。

3) 調用訓練資料集訓練模型

```
SIGNModel.fit(X_train, Y_train, epochs = 10, batch_size=16 )
```

4) 測試訓練之模型

```
preds = SIGNModel.evaluate(X_test, Y_test, batch_size= 128)
```

顯示其測試集之損失和預測精度

```
print("SIGNModel")
print ("Loss = " + str(preds[0]))
print ("Test Accuracy = " + str(preds[1]))
```

若是想更進一步了解 model.compile(...)、model.fit(...) 和 model.evaluate(...) 及其參數，請參閱 Keras 官方文檔 <https://keras.io/>

如果您尚未達到 75% 的準確度，可以嘗試以下方法來達到此目的：

1) 更改 CNN 的擷取特徵數目、特徵擷取時的視窗大小等等

```
X = Conv2D(32, (3, 3), strides = (1, 1), name = 'conv0')(X)
```

2) 在 CNN 之後使用 Maxpool 模塊，降低高和寬的維度

3) 更改 Optimizer，例如：SGD、RMSprop、Adagrad、Adam 等等，詳情參閱 Keras 官網

4) 如果模型運行困難，並且遇到內存問題，請減小 batch_size 處理

3. 總結:

Keras 是一個快速模型製作的工具，它可以嘗試不同的模型架構，若是使用 keras 實現並評估，請記住 keras 中編寫代碼的四個步驟 1) 創建模型、2) 編譯模型、3) 訓練模型、4) 測試模型。

4. 圖片測試:

恭喜完成本次作業，可以加入自己的測試照片來驗證，代碼如下：

```
img_path = 'images/test_1.jpg'
### END CODE HERE ###
img = image.load_img(img_path, target_size=(64, 64))
imshow(img)

x1 = image.img_to_array(img)
x2 = np.expand_dims(x1, axis=0)
x3 = preprocess_input(x2)
x_out = np.argmax(SIGNModel.predict(x3))
print('The figure is',x_out)
```

Img_path: 圖片的位置

Image.load_img: 讀取圖片並轉換成特定大小

Imshow(): 顯示圖片

Image.img_to_array: 圖片轉成像素矩陣

np.expand_dims: 將資料展平，符合模型的輸入

preprocess_input: keras 的圖片預處理

The figure is 1

